

• OPENHARMONY TECH FORUM · GOSIM PARIS 2026

Lowering **every** barrier.

How Oniro brings devices, apps, and
developers to OpenHarmony.

SPEAKER

Francesco Pham · Huawei

AFFILIATION

Eclipse Oniro · Project Lead

DATE

May 2026 · Paris

OpenHarmony's adoption isn't blocked by **technology**.

The kernel is mature. The frameworks are production-grade. Hundreds of products already ship on it.

The blockers are **barriers** — between the platform and the hardware people own, the codebases developers maintain, and the apps users open every day.

This talk is about what it takes to remove each one.

Three barriers stand between OpenHarmony and global adoption.

1

Hardware.

You can't buy a phone that runs it — outside specific markets. The platform is locked to reference hardware.

2

Developer.

Your existing codebase doesn't run on it. Rewriting in ArkTS is not realistic for millions of apps.

3

User.

The apps people actually use aren't there. Without them, the platform doesn't matter to anyone.

Section 01

Devices.

Running OpenHarmony on commercial mobile hardware
that was never designed for it.

OpenHarmony covers three device classes — phones live in **Standard**.



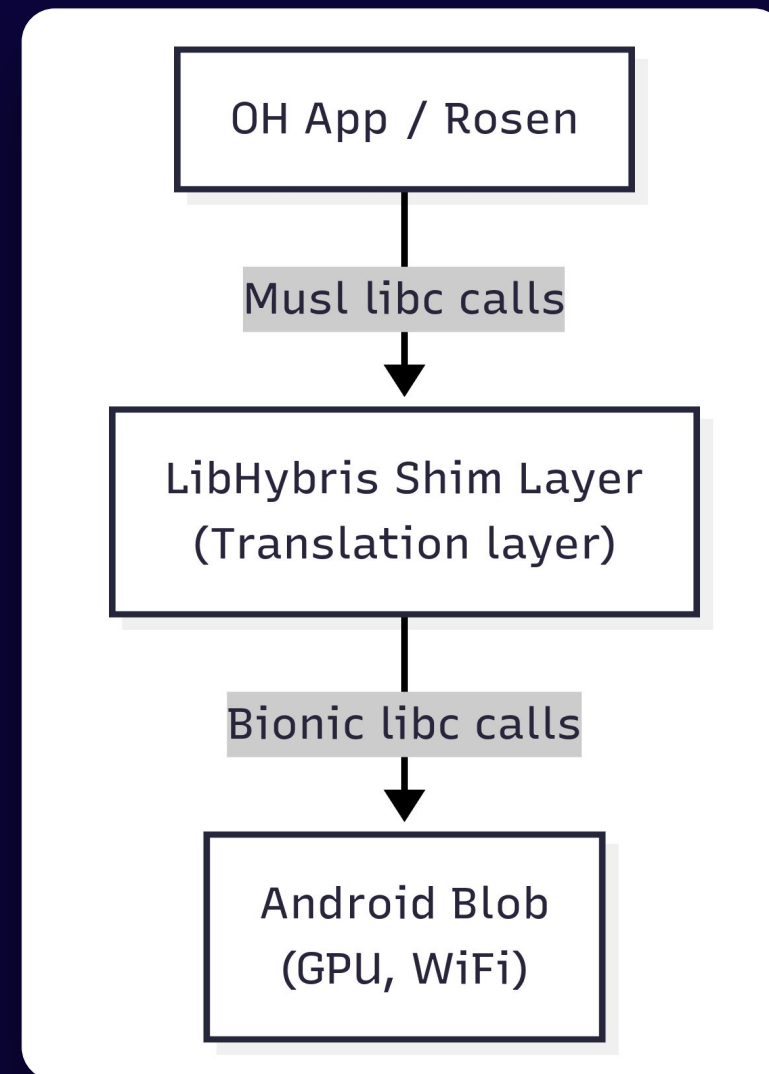
MINI · SMALL · STANDARD — ONE PLATFORM, THREE CLASSES

The platform was designed for the breadth of IoT — wearables to fridges. Phones sit at the top of that range, and that's where the hardware problem gets hardest.

Mobile SoC vendors ship drivers for Android — closed binary blobs, linked against Bionic libc. They will not be open-sourced, and reverse-engineering every chipset is a generational project.

Without a way to run that hardware **today**, the platform reaches only devices its sponsors ship.

LibHybris: a bridge to commercial silicon.



MUSL ⇒ BIONIC SHIM, ISOLATED IN LXC

OpenHarmony userspace is built on musl libc. Vendor blobs were linked against Bionic. The shim translates between them, call by call.

An **LXC container** isolates the Android userspace those drivers depend on — so vendor code runs in its expected environment without contaminating the host.

A pragmatic workaround, not an ideology. The blobs remain black boxes — and devices ship today.



WHAT WORKS TODAY ON COMMERCIAL HARDWARE

A working stack on devices that were never meant to run this.



VOLLA PHONE + TABLET · ONIRO RUNNING

Graphics

Mali GPU via
vendor blobs ·
compositor & HDI
on top

WORKING Display & Touch

Panel,
brightness,
multi-touch input
through HDF

WORKING

Audio

ALSA userspace +
HDF Audio HAL —
playback &
capture

WORKING WiFi

Vendor firmware
via libhybris
bridge · DHCP,
association

WORKING

Power management

Suspend/resume,
charging, battery
reporting

WORKING Cellular & Camera

Modem and ISP
integration in
progress

ONGOING

Section 02

Frameworks.

Bring your codebase. Don't rewrite it.

THE DEVELOPER BARRIER

Two paradigms. Two bridges. Thousands of apps.

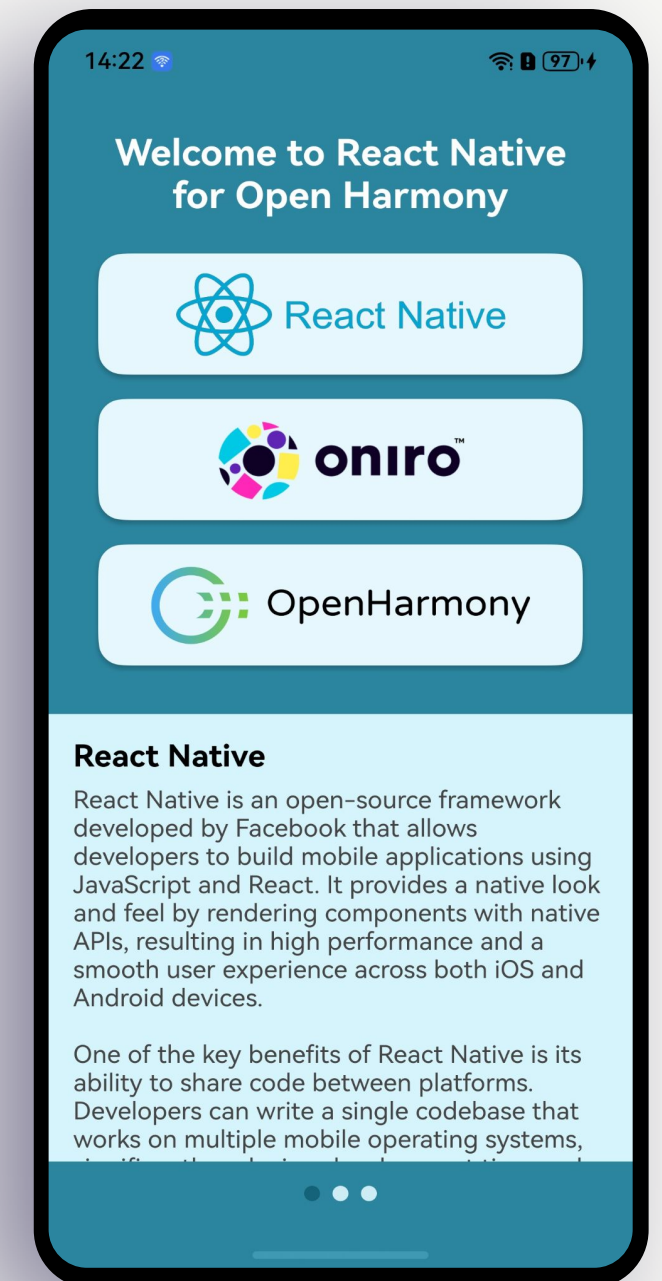
Expecting the global developer base to rewrite in ArkTS is not realistic.

So we picked the two cross-platform philosophies that already cover most of modern app development:

Capacitor for the web-native paradigm — wrap a WebView, expose plugins, ship existing web apps.

Tauri for the compiled-native paradigm — Rust backend, WebView frontend, performance and memory safety.

Together, they unlock thousands of existing apps without a rewrite. **React Native** is on the same trajectory.

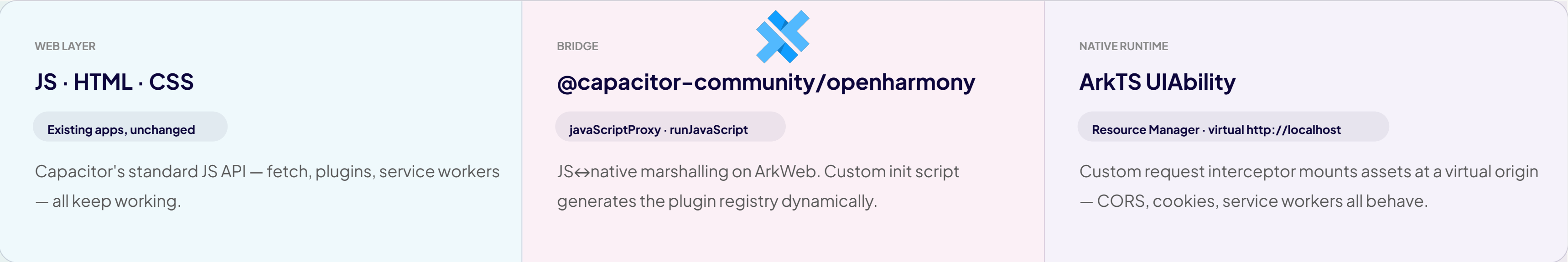


REACT NATIVE FOR OPENHARMONY



CAPACITOR · THE WEB-NATIVE BRIDGE

Web apps, on ArkWeb.



ArkWeb's strict CORS model would break `file://` loading. The virtual-server pattern mimics Capacitor's Android behaviour without weakening the OS security model.

Rust on OpenHarmony — for the first time at framework level.

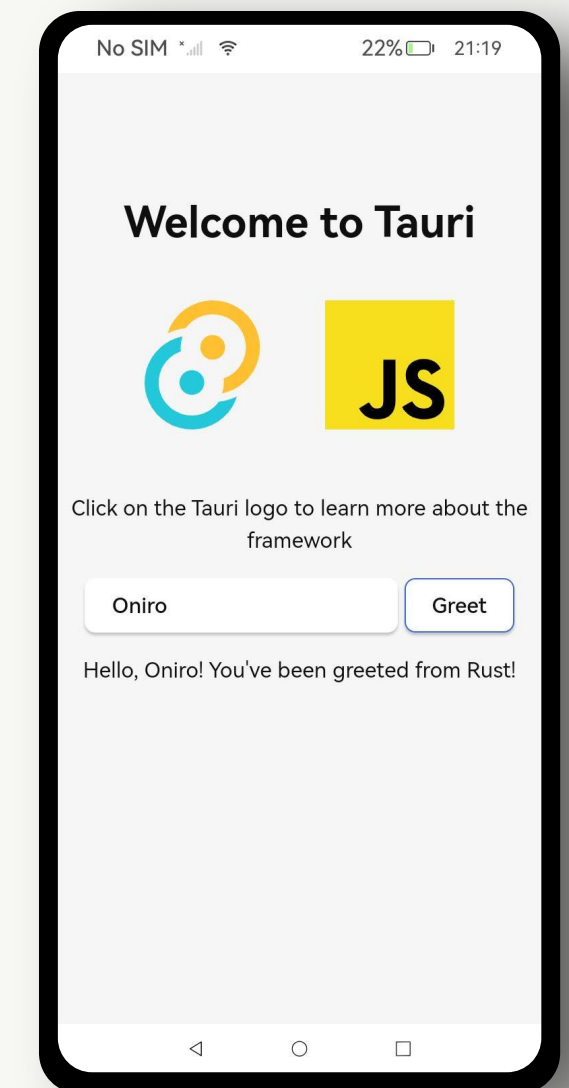
The inversion-of-control problem:

UIAbility owns the process — not Rust *main()*.

On desktop, the Rust process is the master. On OpenHarmony, the OS Ability Manager is. The Rust core has to live as a guest library.

The pattern: Window Adoption via NAPI. The ArkTS *UIAbility* creates the window stage; Tauri's Rust core is loaded as a native library and **adopts** the existing window — taking over rendering and event handling without owning the process.

Bidirectional NAPI bindings carry events and commands across the boundary. Rust gets the deep system access it needs; OpenHarmony keeps process control.



TAURI STARTER APP · GREETED FROM RUST

DEVELOPER EXPERIENCE IS THE MULTIPLIER

The commands work the way developers **expect**.

CAPACITOR

```
$ npx cap add openharmony  
$ npx cap sync openharmony  
  
# scaffolds the native project,  
# copies web assets, generates  
# ArkTS plugin registration.
```

TAURI

```
$ cargo tauri ohos init  
$ cargo tauri ohos build  
  
# unified build, deploy and  
# live-logging – same workflow  
# developers already know.
```

Tooling is often an afterthought. We prioritized it alongside the runtime — because developer accessibility is what makes a port viable.

Lichess running on OpenHarmony — without a rewrite.



Lichess is a heavy Capacitor app: real-time SVG rendering, persistent storage, sound playback, complex network graph.

It's the kind of production-grade software that exposes every weak point in a cross-platform port.

It runs unmodified on OpenHarmony. Same JS bundle, same plugin contract, same expectations. The bridge is real — not a toy demo.

Section 03

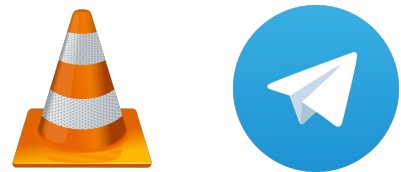
Apps.

Making the platform usable — production-grade, not toy demos.

Apps reach the platform through any of **three** doors.

A

Native ArkTS



First-class apps written for the platform. **VLC** is the flagship — production-grade native, not a demo.

B

Cross-platform via the bridges



Capacitor and Tauri apps from existing codebases. Plus React Native, with Flutter and Kotlin Multiplatform under research.

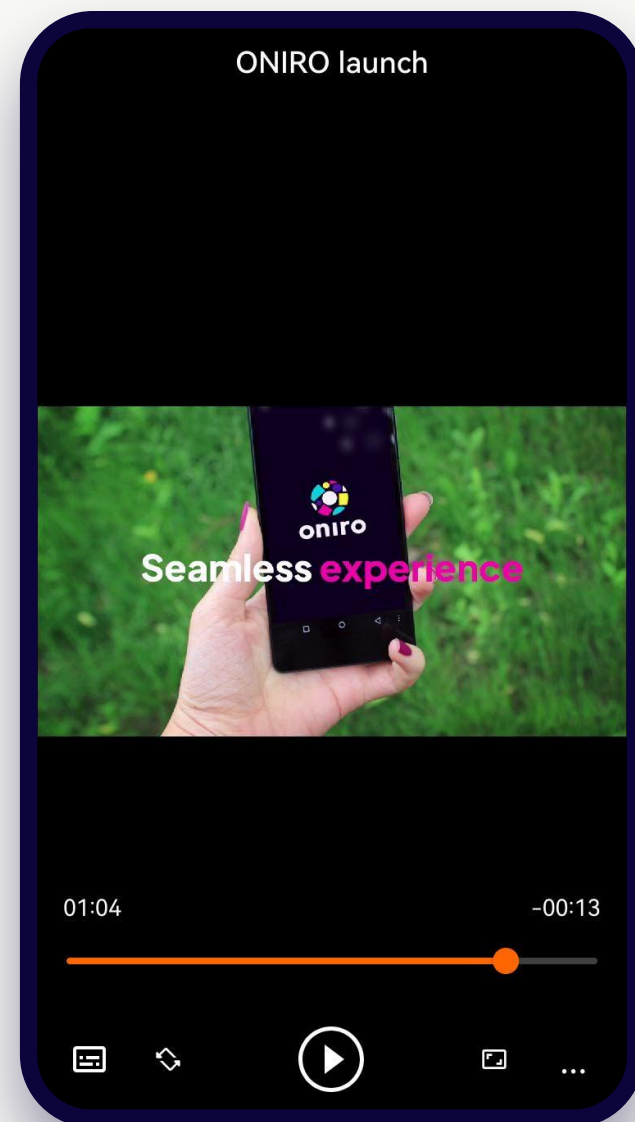
C

Distribution



The **Oniro App Store** — F-Droid-shaped, open-source-first, no gatekeeper.

VLC on Oniro: a 25-year-old C/C++ engine, ported end-to-end.



VLC is the proof point. Not a demo — the real *libVLC* core, ~400 *plugins*, native ArkTS UI on top. Same architecture VideoLAN uses on Android.

THE PORTING STACK — BOTTOM-UP

01 Contrib system patched

FFmpeg, dav1d, x264/x265, freetype, libass, harfbuzz cross-built. config.sub taught the linux-ohos* triplet.

03 NAPI replaces JNI

Node-API wrappers for `media_player_t`.
Thread-safe events via `napi_threadsafe_function`.

05 Custom aout — OHAudio renderer

Bridges VLC's push-based `Play()` to OHAudio's pull-based `OnWriteData` via a mutex-guarded block chain.
Verified on device: **.mp4 .mkv videos** playing — hardware GLES2 video, OHAudio output, ~400 plugins loaded.

02 libVLC core, OHOS triplet

`--host=aarch64-linux-ohos`. musl has no `pthread_cancel` — stubbed.

04 Custom vout — EGL/GLES2

`SurfaceId` → `OHNativeWindow` → EGL surface, hardware GLES2 path.



DOOR C · DISTRIBUTION WITHOUT A GATEKEEPER

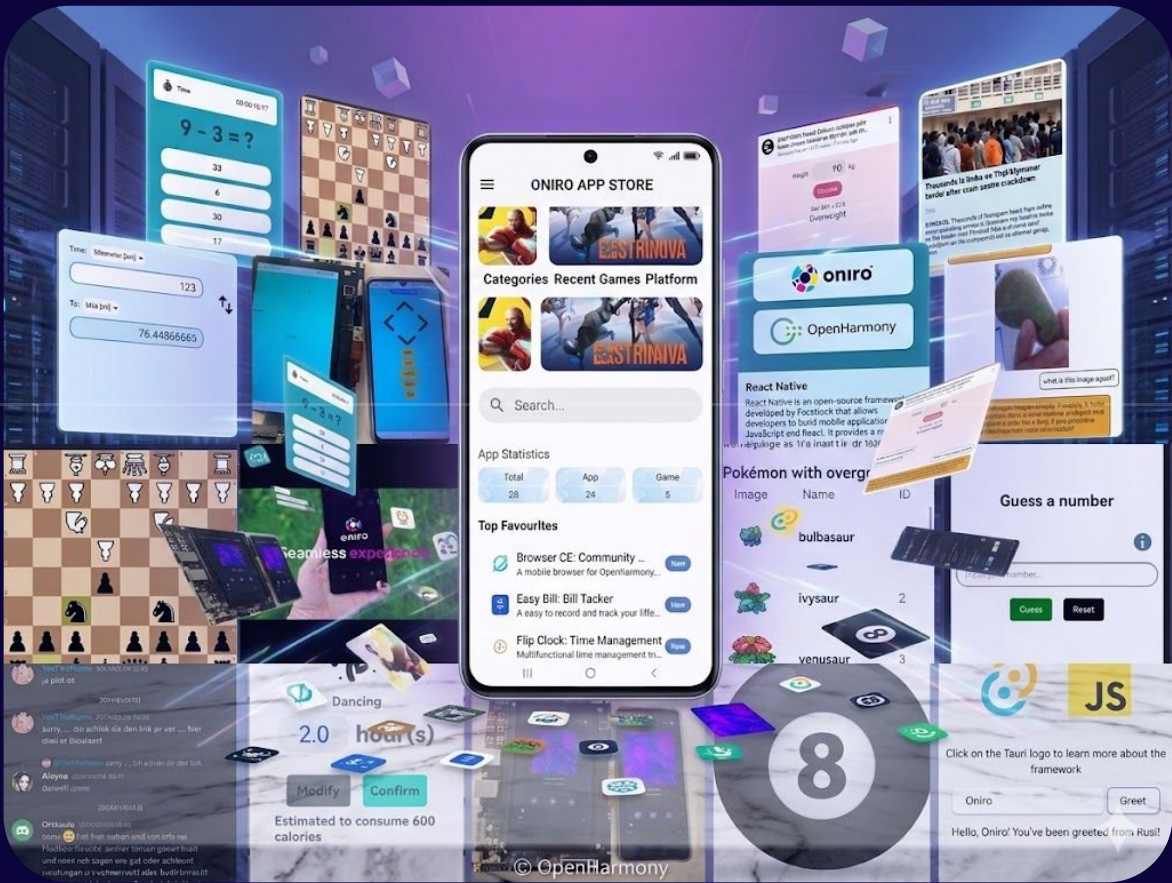
An app store shaped by the community — not by a single vendor.

The **Oniro App Store** follows the F-Droid model: open catalog, reproducible builds, no proprietary review process, no revenue cut.

It is not a clone of Google Play. The point is precisely that distribution doesn't depend on a gatekeeper.

ALREADY RUNNING ON ONIRO

VLC		Native ArkTS · libVLC core	NATIVE
Lichess		React + Capacitor, unmodified	CAPACITOR
Telegram		Native port — messaging baseline	NATIVE



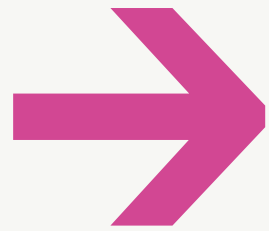
Section 04

The how.

AI-orchestrated development is the method that made the scope possible.

COMPRESSION

Months



Weeks.

Two production-ready framework ports. One developer.

The **three-phase pattern** behind it:

0

Deep research · context

A 40,000-word strategy doc — the AI's single source of truth.

1

Atomic decomposition

Concrete, verifiable tasks. Not "port Capacitor" — "implement onInterceptRequest."

2

Self-healing verification loop

Build → deploy → analyze HiLog → diagnose → fix — autonomously, 5–10 cycles.

Director. Not author.

The senior engineer's role shifts: from writing syntax to setting strategy and verifying outcomes.

System intuition still essential. The AI scaffolds and translates; humans diagnose the deep platform quirks the docs don't cover.

This is the strategic implication for FOSS: it changes the math on what a small open-source team can take on.

THE DOORS ARE OPEN

Three asks — one per pillar.

HARDWARE PORTERS

Pick a device. Bring the bring-up. We'll meet you at the bridge.

APP & FRAMEWORK DEVS

Try Capacitor or Tauri on Oniro. Your codebase already runs.

EVERYONE ELSE

Try the apps. File issues. Tell us what's missing.

Digital sovereignty isn't won by regulation alone — it's won by technical alternatives that work today. An open mobile platform is part of that infrastructure. The community is ready to build it.

**Contribute to Oniro development.
Let's create an open, secure mobile
ecosystem together.**

CODE

github.com/eclipse-oniro4openharmony

DOCS

docs.oniroproject.org

Think Global and Code Local